

NYCU DLP Lab3

1. Overview of your lab 3

- Objective:

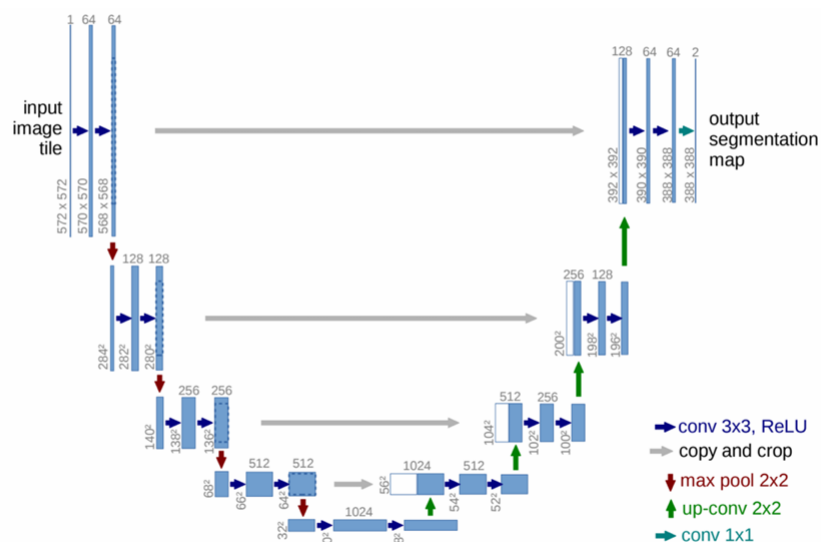
In this lab, we will explore the fascinating field of semantic segmentation using deep learning techniques. Specifically, we will focus on binary semantic segmentation, where the goal is to classify each image pixel as belonging to the foreground (object of interest) or the background.

- DataSet:

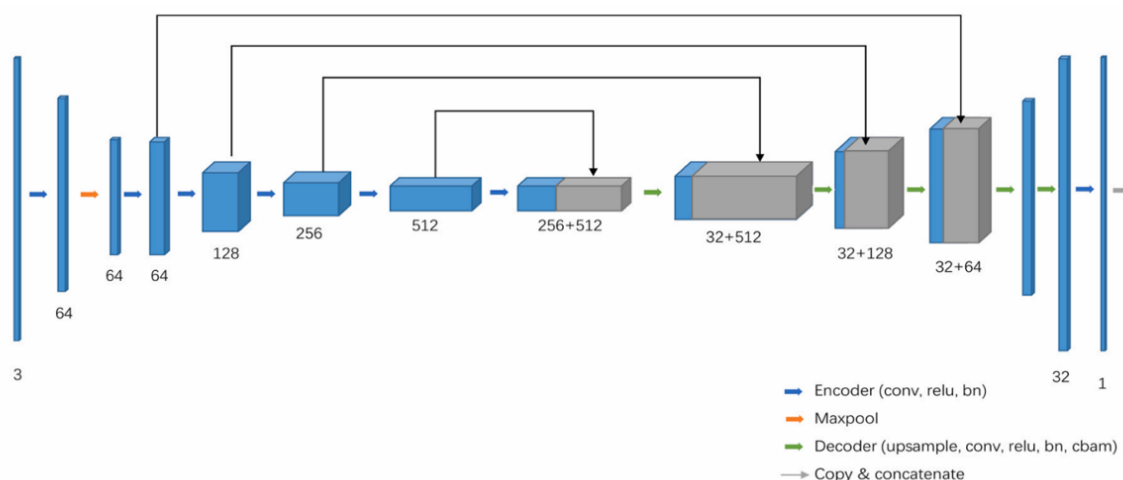
The Oxford-IIIT Pet Dataset consists of images of cats and dogs, annotated with pixel-level labels for the pet regions. We will work with this dataset to train and evaluate their segmentation models.

- Model : using Unet and ResNet34_Unet to compare two model performance

UNet



ResNet34 (Encoder) + UNet (Decoder)



2. Implementation Details

A.Details of your training, evaluating, inferencing code

- Training code
 - select the training model via args.model
 - using Adam as optimizer with weight decay 1e-4 to prevent overfitting
 - with lr = 1e-4 , batch_size: 16
 - using BCEWithLogitsLoss as loss function , which is suitable for binary classification problem
 - During training with record the loss , whereas when testing we use Dice Score to express the model's performance
 - After the entire training process, I save the loss and Dice score as numpy arrays to plot the graphs.

```
import argparse
import torch
import torch.nn as nn
import os
import numpy as np

from models.unet import Unet
from models.resnet34_unet import ResNet34_UNet
from oxford_pet import load_dataset
from torch.utils.data import DataLoader
from utils import dice_score
from tqdm import tqdm

def train(args,model,save_path):
    # implement the training function here

    # make the dir to save different model
    os.makedirs(f"{save_path}/{args.model}", exist_ok=True)
    # "cuda" only when GPUs are available.
    device = "cuda" if torch.cuda.is_available() else "cpu"
    #Define model
    model = model.to(device)
    #Prepare Data
    train_data = load_dataset(data_path=args.data_path ,mode='train')
    train_loader = DataLoader(train_data,batch_size=args.batch_size,shuffle=True,pin_memory=True,num_workers=1)

    val_data=load_dataset(data_path=args.data_path ,mode='test')
    val_loader=DataLoader(val_data,batch_size=args.batch_size,shuffle=False,pin_memory=True,num_workers=1)

    #loss function
    criterion = nn.BCEWithLogitsLoss() #適用二分類問題 將model映射成[0,1] 透過sigmoid function
    #optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr=args.learning_rate, weight_decay=1e-4)
    # put dice scores and losses
    losses=[]
    dice_scores=[]

    # Define best scores
    best_scores=0
    for epoch in range(args.epochs):
        train_loss=0
        model.train()
        for sample in tqdm(train_loader): # (資料集個數) *0.9 / batch_size
            #transfer to GPU
            image, mask = sample["image"].to(device), sample["mask"].to(device)

            pred = model(image.to(dtype=torch.float32)) #already flatten

            loss = criterion(pred,mask)

            train_loss+=loss.item()

            # Gradients stored in the parameters in the previous step should be cleared out first.
            optimizer.zero_grad()

            # Compute the gradients for parameters.
            loss.backward()

            # Update the parameters with computed gradients.
            optimizer.step()

        train_loss = train_loss / len(train_loader)
```

```

model.eval()
result=0

##### eval here #####
with torch.no_grad():
    for sample in val_loader:
        image, mask = sample["image"].to(device), sample["mask"].to(device)
        pred = model(image.to(dtype=torch.float32))
        pred=torch.sigmoid(pred)
        ##### calculate dice score
        result += dice_score(pred,mask)

    score= result / len(val_loader)

dice_scores.append(score)
losses.append(train_loss)
print(f"[ Train | {epoch + 1:03d}/{args.epochs:03d} ] loss = {train_loss:.5f},dice_score = {score:.2f}")
if score>best_scores:
    best_scores=score
    torch.save(model.state_dict(), f"{save_path}/model_{args.model}_{score:.2f}.pth")
    print(f"Save the model with score{score:.2f},The destination is {save_path}")

#os.makedirs(save_path, exist_ok=True)

#torch.save(model.state_dict(), f"{save_path}/model.pth")
#print(f'model save to {save_path}')
#turn into numpy file to plot the graph
#np.save(f"{save_path}/{args.model}/losses_witoutDA_{args.model}.numpy", losses)
#np.save(f"{save_path}/{args.model}/dice_scores_witoutDA_{args.model}.numpy", dice_scores)
#assert False, "Not implemented yet!"

def get_args():
    parser = argparse.ArgumentParser(description='Train the UNet on images and target masks')
    parser.add_argument('--data_path', default= './dataset/oxford-iiit-pet',type=str, help='path of the input data')
    parser.add_argument('--epochs', '-e',type=int, default=25, help='number of epochs')
    parser.add_argument('--batch_size', '-b',type=int, default=16, help='batch size')
    parser.add_argument('--learning-rate', '-lr',type=float, default=1e-4, help='learning rate')
    parser.add_argument('--model',type=str, default="Unet",choices=['Unet', 'ResNet34_UNet'], help='Unet or ResNet34_UNet ')

    return parser.parse_args()

if __name__ == "__main__":
    args = get_args()
    #model = Unet(in_ch=3,out_ch=1)
    if(args.model == "Unet"):
        print("Training Unet")
        model = Unet(in_ch=3,out_ch=1)
    elif(args.model=="ResNet34_UNet"):
        print("Training ResNet34_UNet")
        model=ResNet34_UNet(in_ch=3,out_ch=1)
    train(args,model=model,save_path='./saved_models')

```

- Evaluating code:
 - using dice score to evaluate the model's performance

```

import torch
from utils import dice_score

def evaluate(model, data):
    # implement the evaluation function here
    # "cuda" only when GPUs are available.
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = model.to(device)
    model.eval()
    with torch.no_grad():
        result=0
        for sample in data:
            image, mask = sample["image"].to(device), sample["mask"].to(device)
            pred = model(image.to(dtype=torch.float32))
            ##### calculate dice score
            result += dice_score(pred,mask)
        score= result / len(data)

```

```

    return score
    #assert False, "Not implemented yet!"

```

- Inferencing code
 - The function testing uses the Dice score to evaluate the model's performance. In both models, I can achieve a Dice score of 0.9.
 - The function show I can save the origin image and pred image to compare the result.
 - The function combine can show the result after mix

```

import argparse
import torch
import torch.nn as nn
import os
import numpy as np

from PIL import Image
from models.resnet34_unet import ResNet34_UNet
from models.unet import Unet
from oxford_pet import load_dataset
from torch.utils.data import DataLoader
from utils import dice_score

def get_args():
    parser = argparse.ArgumentParser(description='Predict masks from input images')
    parser.add_argument('--Unet', default='./saved_models/model_Unet_0.91.pth', help='Unet')
    parser.add_argument('--ResNet34_Unet', default='./saved_models/model_ResNet34_Unet_0.91.pth', help='ResNet34_Unet')
    parser.add_argument('--data_path', type=str, default='./dataset/oxford-iiit-pet', help='path to the input data')
    parser.add_argument('--batch_size', '-b', type=int, default=1, help='batch size')
    parser.add_argument('--model_path', type=str, default='./saved_models/model_Unet0.91pth', help='The model path ')
    parser.add_argument('--model', type=str, default="UNet", choices=['UNet', 'ResNet34_Unet'], help='Unet or ResNet34_Unet ')
    return parser.parse_args()

def test(args):
    # "cuda" only when GPUs are available.
    device = "cuda" if torch.cuda.is_available() else "cpu"

    #load the model
    if(args.model=='UNet'):
        print("Using UNet")
        model = Unet(in_ch=3,out_ch=1).to(device)
        model.load_state_dict(torch.load(args.model_path))
    elif(args.model=='ResNet34_Unet'):
        print("Using ResNet34_Unet")
        model = ResNet34_UNet(in_ch=3,out_ch=1).to(device)
        model.load_state_dict(torch.load(args.model_path))

    model.eval()
    #load the test data
    test_data = load_dataset(data_path=args.data_path,mode='test')
    test_loader = DataLoader(test_data,batch_size=args.batch_size,shuffle=True,pin_memory=True)

    result=0

    with torch.no_grad():
        for sample in test_loader:
            image, mask = sample["image"].to(device), sample["mask"].to(device)
            pred = model(image.to(dtype=torch.float32))
            #output = model(image.to(dtype=torch.float32))
            pred=torch.sigmoid(pred)
            ##### calculate dice score#####
            result += dice_score(pred,mask)

        score= result / len(test_loader)
        print(f"The dice score is {score:.2f}")

    return score

# show the origin and pred picture
def show(args): # plot the grapg with origin and not origin
    #make the dirs to put the image
    os.makedirs("../img",exist_ok=True)
    # "cuda" only when GPUs are available.

    device = "cuda" if torch.cuda.is_available() else "cpu"
    test_data = load_dataset(data_path=args.data_path,mode='test')
    test_loader = DataLoader(test_data,batch_size=args.batch_size,shuffle=True,pin_memory=True)

```

```

data_iter = iter(test_loader)
batch = next(data_iter)
image, mask = batch["image"], batch["mask"]
#convert to numpy
image_np = image.numpy()
# eliminate the first dim
image_np = np.squeeze(image_np, axis=0)
# change CHW -> HWC
image_np = np.moveaxis(image_np, 0, 2)

image_np = image_np.astype(np.uint8)

original_image = Image.fromarray(image_np)
original_image.save(f'../img/original_image.png')

##### mask result #####
#load the model

model1 = Unet(in_ch=3,out_ch=1).to(device)
model1.load_state_dict(torch.load(args.Unet))

model2 = ResNet34_UNet(in_ch=3,out_ch=1).to(device)
model2.load_state_dict(torch.load(args.ResNet34_Unet))

##### predict the result (Unet) #####
image, mask = batch["image"].to(device), batch["mask"].to(device)
with torch.no_grad():
    pred = model1(image.to(dtype=torch.float32))
    pred = torch.sigmoid(pred) #map to zero or one

pred = (pred > 0.5).float()
pred_np = pred.cpu().numpy()
##### handle dim#####
#print(pred_np.shape) #(1,65536)
pred_np=pred_np.reshape(1,1,256,256) #(1,1,256,256)
pred_np = np.squeeze(pred_np, axis=0) #(1,256,256)
pred_np = np.moveaxis(pred_np, 0, 2) #(256,256,1)
pred_np = pred_np.astype(np.uint8)
pred_np = np.squeeze(pred_np, axis=2)# (256,256)
##### coloring #####

green = [0, 255, 0] # greeb
blue = [0, 0, 255] # blue
color_image = np.zeros((256, 256, 3), dtype=np.uint8)
color_image[pred_np ==1] = green # <0.5 -> green
color_image[pred_np == 0] = blue # >=0.5 blue

#print(type(pred_np))
original_image = Image.fromarray(color_image)
original_image.save(f'../img/pred_image_Unet.png')

##### predict the result (ResNet34) #####
image, mask = batch["image"].to(device), batch["mask"].to(device)
with torch.no_grad():
    pred = model2(image.to(dtype=torch.float32))
    pred = torch.sigmoid(pred) #map to zero or one

pred = (pred > 0.5).float()
pred_np = pred.cpu().numpy()
##### handle dim#####
#print(pred_np.shape) #(1,65536)
pred_np=pred_np.reshape(1,1,256,256) #(1,1,256,256)
pred_np = np.squeeze(pred_np, axis=0) #(1,256,256)
pred_np = np.moveaxis(pred_np, 0, 2) #(256,256,1)
pred_np = pred_np.astype(np.uint8)
pred_np = np.squeeze(pred_np, axis=2)# (256,256)
#print(type(pred_np))
original_image = Image.fromarray(color_image)
original_image.save(f'../img/pred_image_ResNet34_Unet.png')

#combine the origin and mask image
def combine(pic1,pic2): # merge the two picture
    #conver the image to RGB
    image1 = Image.open(pic1).convert('RGB')
    image2 = Image.open(pic2).convert('RGB')

```

```

img =Image.blend(image1, image2, alpha=0.5)
# Extract the base name of the second image file
pic2_basename = os.path.basename(pic2)

# Determine the output file name based on pic2's name
if 'pred_image_ResNet34_Unet.png' in pic2_basename:
    output_filename = 'blend_image_ResNet34_Unet.png'
elif 'pred_image_Unet.png' in pic2_basename:
    output_filename = 'blend_image_Unet.png'
else:
    output_filename = 'blend_image.png' # Default name if no match

# Save the blended image with the determined name
output_path = os.path.join('../img', output_filename)
img.save(output_path)
print(f"Combined image saved as {output_path}")

if __name__ == '__main__':
    args = get_args()
    test(args)
    #show(args)
    #combine('../img/original_image.png', '../img/pred_image_Unet.png')
    #combine('../img/original_image.png', '../img/pred_image_ResNet34_Unet.png')
    #assert False, "Not implemented yet!"

```

B.Details of your model (UNet & ResNet34_Unet)

- Unet:
 - I use ConvBlock as the basic building block for the UNet, which contains two times of conv.
 - **Encoder (Downsampling Path)**
 - **Convolution Block (ConvBlock):** Each layer consists of two convolution layers followed by a ReLU activation function, then batch normalization.
 - **Max Pooling Layer:** After each convolution block, a max pooling layer is used for downsampling, reducing spatial dimensions and increasing the number of feature maps.
 - **Decoder (Upsampling Path)**
 - **Upsampling:** Increase spatial dimensions through upsampling (e.g., transposed convolution or upsampling layers).
 - **Convolution Block (ConvBlock):** Each layer consists of two convolution layers followed by a ReLU activation function, then batch normalization.
 - **Skip Connections:** Concatenate the feature maps from the corresponding encoder layer with the upsampled feature maps in the decoder to retain detailed information.

```

# Implement your UNet model here
import torch
import torch.nn as nn
from torch import Tensor
from torch.utils.data import TensorDataset

#assert False, "Not implemented yet!"

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        output = self.conv(x)
        return output

#做up conv decoder將特徵圖放大
#這種設計的主要目的是保留高分辨率的信息，
#這些信息在downsample過程中可能會丟失。
#通過(skip connection), U-Net
#能夠在decode過程中利用來自encoder的細節信息，從而提高最終的分割結果。
class Unet(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        ##### Conv encoder block#####

        self.conv1 = ConvBlock(in_ch, 64)
        self.pool1 = nn.MaxPool2d(kernel_size=(2, 2))

```

```

self.conv2 = ConvBlock(64, 128)
self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

self.conv3 = ConvBlock(128, 256)
self.pool3 = nn.MaxPool2d(kernel_size=(2, 2))

self.conv4 = ConvBlock(256, 512)
self.pool4 = nn.MaxPool2d(kernel_size=(2, 2))

self.conv5 = ConvBlock(512, 1024)

##### up conv decoder block#####
self.upconv1=nn.ConvTranspose2d(1024,512, kernel_size=(2,2),stride=2) # H,W expand 兩倍
self.deconv1 = ConvBlock(1024,512)

self.upconv2=nn.ConvTranspose2d(512,256, kernel_size=(2,2),stride=2)
self.deconv2 = ConvBlock(512,256)

self.upconv3=nn.ConvTranspose2d(256,128, kernel_size=(2,2),stride=2)
self.deconv3= ConvBlock(256,128)

self.upconv4=nn.ConvTranspose2d(128,64, kernel_size=(2,2),stride=2)
self.deconv4= ConvBlock(128,64)

self.deconvfinal = nn.Conv2d(64,out_ch,kernel_size=(1,1))

def forward(self, x):
    ##### Encoder part #####
    # Initial input shape: [16, 3, 256, 256]
    c1 = self.conv1(x) # [16, 64, 256, 256]

    p1 = self.pool1(c1)

    c2 = self.conv2(p1)

    p2 = self.pool2(c2)

    c3 = self.conv3(p2)

    p3 = self.pool3(c3)

    c4 = self.conv4(p3)

    p4 = self.pool4(c4)

    c5 = self.conv5(p4)

    #####crop and paste#####

    up1 = self.upconv1(c5) #[16, 512, 32, 32]

    x1 = torch.cat((c4, up1), dim=1)#[16, 1024, 32, 32]

    x2 = self.deconv1(x1)#[16, 512, 32, 32]

    up2 = self.upconv2(x2)#[16, 256, 64, 64]

    x3 = torch.cat((c3, up2), dim=1)#[16, 512, 64, 64]

    x4 = self.deconv2(x3)#[16, 256, 64, 64]

    up3 = self.upconv3(x4)#[16, 128, 128, 128]

    x5 = torch.cat((c2, up3), dim=1)#[16, 256, 128, 128]

    x6 = self.deconv3(x5)#[16, 128, 128, 128]

    up4 = self.upconv4(x6)#[16, 64, 256, 256]

    x7 = torch.cat((c1, up4), dim=1)#[16, 128, 256, 256]

    x8 = self.deconv4(x7)#[16, 64, 256, 256]

```

```
#####final conv#####

output = self.deconvfinal(x8)#[16, 1, 256, 256]

output = output.flatten(start_dim=1)#[16, 65536]

return output
```

- ResNet34_Unet:

- **Encoder (Downsampling Path):**

- **ResNet34 Backbone:** Uses the ResNet34 architecture as the backbone for feature extraction, including:
 - **Convolution Layer and ReLU Activation:** Initial convolution layer and ReLU activation function.
 - **Residual Blocks:** Each residual block consists of convolution layers and skip connections for deeper feature learning.
 - **Max Pooling Layer:** Max pooling layers are used for downsampling, reducing spatial dimensions and increasing the number of feature maps.

- **Decoder (Upsampling Path):**

- **Upsampling:** Increase spatial dimensions through upsampling (e.g., transposed convolution or upsampling layers).
- **Convolution Block (ConvBlock):** Each layer consists of two convolution layers followed by a ReLU activation function, then batch normalization.
- **Skip Connections:** Concatenate the feature maps from the corresponding encoder layer with the upsampled feature maps in the decoder to retain detailed information

```
# Implement your ResNet34_Unet model here
import torch
import torch.nn as nn
import torch.nn.functional as F

from torch import Tensor
from torch.utils.data import TensorDataset

#assert False, "Not implemented yet!"

#multilayer layer of 3X3 kernel
#Residual learning

# block for ResNet34(解決了網路deeper ,效果比較差的問題,學習F(x)+x)
# stride =1 代表原圖大小不變
# stride =2 代表原圖大小變成一半
class BasicBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=(3,3), stride=stride, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=(3,3), stride=1, padding=1),
            nn.BatchNorm2d(out_ch),
        )
        #down 代表需不需要降維
        if stride!=1 or in_ch != out_ch: #stride !=1 或 in_ch != out_ch 代表要降低維度
            self.down = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, kernel_size=1, stride=stride),
                nn.BatchNorm2d(out_ch)
            )
        else:
            self.down=None
    def forward(self, x):
        out = self.conv(x)
        if self.down is not None:
            residual=self.down(x)
        else:
            residual=x

        out = residual + out
        out = F.relu(out)
        return out

class EncoderBlock(nn.Module):
    def __init__(self, in_ch, out_ch, number): # 需要幾個 BasicBlock
        super().__init__()
        layers = []
        layers.append(BasicBlock(in_ch, out_ch, 2)) #變一半
        for _ in range(1, number):
            layers.append(BasicBlock(out_ch, out_ch, 1)) #
        self.blocks = nn.Sequential(*layers)
```

```

def forward(self, x):
    out = self.blocks(x)
    return out

##### block for Unet#####
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        output = self.conv(x)
        return output

class DecoderBlock(nn.Module):
    def __init__(self, in_ch, out_ch, up_in_ch=None, up_out_ch=None):
        super().__init__()
        if up_in_ch==None:
            up_in_ch=in_ch
        if up_out_ch==None:
            up_out_ch=out_ch

        self.up = nn.ConvTranspose2d(up_in_ch, up_out_ch, kernel_size=2, stride=2)

        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=(3,3),padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    # x1-upconv , x2-downconv
    def forward(self, x1, x2):
        x1 = self.up(x1)
        x = torch.cat([x1, x2], dim=1)
        return self.conv(x)

class ResNet34_UNet(nn.Module):
    def __init__(self, in_ch=3, out_ch=1):
        super().__init__()
        self.encoder1=nn.Sequential(
            nn.Conv2d(in_ch,64,kernel_size=(7,7),stride=2,padding=3),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=(3,3),stride=2,padding=1)
        )# H/4
        self.encoder2=EncoderBlock(64,64,3) # H/8
        self.encoder3=EncoderBlock(64,128,4) # H/16
        self.encoder4=EncoderBlock(128,256,6)# H/32
        self.encoder5=EncoderBlock(256,512,3)# H/64
        ##### mid #####
        self.bridge = nn.Sequential(
            nn.Conv2d(512, 1024, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(1024),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.decoder1 = DecoderBlock(1024,512)
        self.decoder2 = DecoderBlock(512,256)
        self.decoder3 = DecoderBlock(256,128)
        self.decoder4 = DecoderBlock(128,64)
        self.decoder5 = DecoderBlock(128,64,64,64)
        self.last = nn.Sequential(
            nn.ConvTranspose2d(64,64, kernel_size=(4,4), stride=4), #變成4倍

```

```

        nn.Conv2d(64, out_ch, kernel_size=(3,3), padding=1, bias=False)
    )
def forward(self, x):
    ##### encoder #####
    # print("Initial input shape:", x.shape) # [16, 3, 256, 256]
    e1 = self.encoder1(x) # [16, 64, 64, 64]
    # print("Encoder1 output shape:", e1.shape)

    e2 = self.encoder2(e1) # [16, 64, 32, 32]
    # print("Encoder2 output shape:", e2.shape)

    e3 = self.encoder3(e2) # [16, 128, 16, 16]
    # print("Encoder3 output shape:", e3.shape)

    e4 = self.encoder4(e3) # [16, 256, 8, 8]
    # print("Encoder4 output shape:", e4.shape)

    e5 = self.encoder5(e4) # [16, 512, 4, 4]
    # print("Encoder5 output shape:", e5.shape)
    ##### mid #####
    mid = self.bridge(e5) # [16, 1024, 2, 2]
    # print("mid output shape:", mid.shape)
    ##### decoder #####
    d1 = self.decoder1(mid, e5) # [16, 512, 4, 4]
    # print(f"d1 shape: {d1.shape}")

    d2 = self.decoder2(d1, e4) # [16, 256, 8, 8]
    # print(f"d2 shape: {d2.shape}")

    d3 = self.decoder3(d2, e3) # [16, 128, 16, 16]
    # print(f"d3 shape: {d3.shape}")

    d4 = self.decoder4(d3, e2) # [16, 64, 32, 32]
    # print(f"d4 shape: {d4.shape}")

    d5 = self.decoder5(d4, e1) # [16, 64, 64, 64]
    # print(f"d5 shape: {d5.shape}")

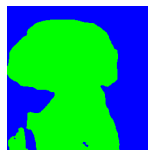
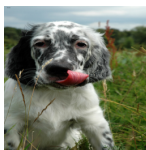
    output = self.last(d5) # [16, 1, 256, 256]
    # print(f"output shape before flatten: {output.shape}")

    output = output.flatten(1) # [16, 65536]
    # print(f"output shape after flatten: {output.shape}")

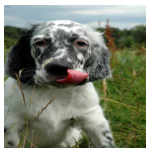
    return output

```

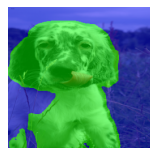
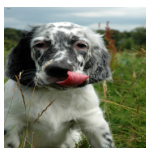
C.The result after predicting



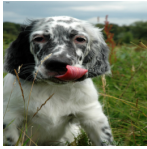
The origin graph versus prediction result of ResNet34_Unet



The origin graph versus prediction result of Unet



The origin graph versus mix result of Unet



The origin graph versus mix result of Unet

We can see that for a simple dataset, the two models appear similar.

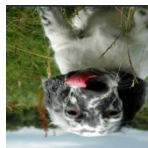
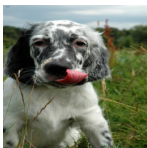
3.Data Preprocessing

A.How you pre processed your data?

- Rotation and Flipping:
 - I randomly select one of four angles (0, 90, 180, 360) to flip the image.
- Cropping
 - I use central cropping with size(200,200) to crop the important region.
- Combining the image mask to dictionary and resize it to 256*256

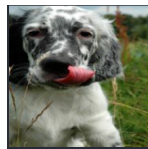
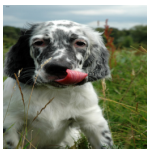
B.What makes your method unique?

- For the rotation results, we use four angles (0, 90, 180, 360) to allow the model to learn different types of data variations
- The rotation increases the diversity of the training data, helping the model to generalize better and reducing the risk of overfitting.



The rotating result versus origin result.

- By cropping the central part of an image, the model can focus on the most relevant features, which are often located in the center.
- Central cropping introduces slight variations in the input images, helping the model to generalize better and reducing overfitting.
- By reducing the size of the input images, central cropping can speed up the training process and reduce computational costs.



The central crop result versus the original result.

C. The data augementation code

```
def trans(**sample):

    #transfer to Image
    img = Image.fromarray(sample["image"]).resize((256, 256), Image.BILINEAR) # type: ignore
    mask = Image.fromarray(sample["mask"]).resize((256, 256), Image.NEAREST) # type: ignore

    ##### rotation the picture #####

    rndn = np.random.random()
    if rndn<0.15:
        transform1=transforms.RandomHorizontalFlip(p=1)
    elif rndn < 0.3:
        transform1=transforms.RandomVerticalFlip(p=1)
    elif rndn<0.45 :
        transform1=transforms.RandomRotation([90,90])
    elif rndn<0.6 :
        transform1=transforms.RandomRotation([270,270])
    else:
        transform1=None

    # do DA
    if transform1 is not None:
        img=transform1(img)
        mask=transform1(mask)
```

```
#####crop the image#####
option=[transforms.CenterCrop(size=(200,200)),None]

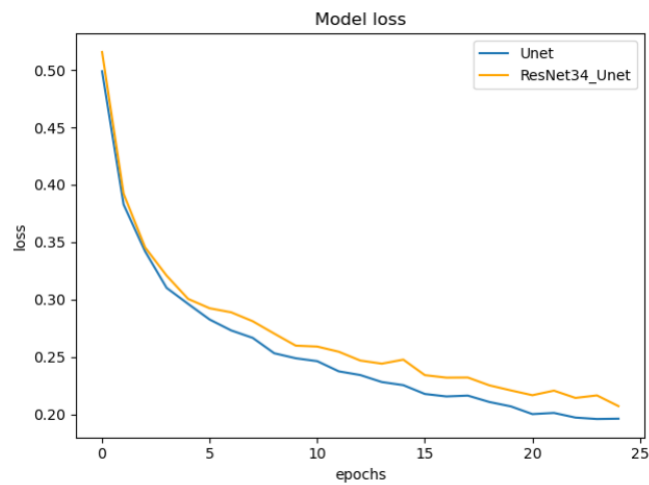
index = random.randint(0, len(option) - 1)
transform2=option[index]

if transform2 is not None:
    img=transform2(img)
    mask=transform2(mask)

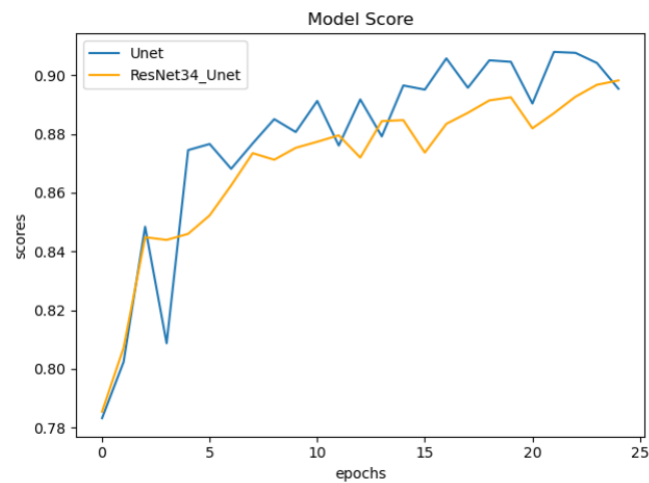
# transfer to np
sample = dict(image= np.array(img), mask= np.array(mask))
#print("doing trans")
return sample
```

A.What did you explore during the training process?

- The loss between two models



- The scores between two models





These two graphs show the performance of the UNet and ResNet34_UNet models during training.

1. Loss Graph (Model Loss):

- Both models' losses decrease rapidly at the beginning of the training, indicating that the models are quickly learning the features of the data.
- As training progresses, the losses gradually stabilize.
- Overall, the UNet's loss is slightly lower than that of the ResNet34_UNet, suggesting that the UNet may have better performance on this dataset.

2. Score Graph (Model Score):

- Both models' scores increase rapidly at the beginning of the training, indicating an improvement in their performance.
- Throughout the training, the UNet's scores fluctuate more but are generally slightly higher than those of the ResNet34_UNet.
- Ultimately, the UNet's score reaches around 0.91, while the ResNet34_UNet's score is slightly lower, close to 0.90.

Overall, these graphs indicate that the UNet performs slightly better than the ResNet34_UNet during training on this dataset.

B. Found any characteristics of the data?

- As the number of epochs increases, the difference between the two models becomes smaller. I believe this may be because the dataset is simple, causing the models to eventually learn similar things. If a more complex dataset is used, the performance difference between the two models would be more noticeable.
- The dice score in the first part can reach very high, I think the dataset is very simple. After the simple DA can reach very good result.

5. Execution command

A. The command and parameters for the training process

- The current directory should be in DL_Lab3_313551110_陳憲驥

```
def get_args():
    parser = argparse.ArgumentParser(description='Train the UNet on images and target masks')
    parser.add_argument('--data_path', default= './dataset/oxford-iiit-pet', type=str, help='path of the input data')
    parser.add_argument('--epochs', '-e', type=int, default=25, help='number of epochs')
    parser.add_argument('--batch_size', '-b', type=int, default=16, help='batch size')
    parser.add_argument('--learning-rate', '-lr', type=float, default=1e-4, help='learning rate')
    parser.add_argument('--model', type=str, default="UNet", choices=['UNet', 'ResNet34_UNet'], help='UNet or ResNet34_UNet ')

    return parser.parse_args()
```



```
python ./src/train.py --model ResNet34_UNet # training ResNet34_UNet
python ./src/train.py --model UNet # training UNet
```

B. The command and parameters for the inference process

- The current directory should be in DL_Lab3_313551110_陳憲驥

```
def get_args():
    parser = argparse.ArgumentParser(description='Predict masks from input images')
    parser.add_argument('--UNet', default='./saved_models/model_UNet_0.91.pth', help='UNet')
    parser.add_argument('--ResNet34_UNet', default='./saved_models/model_ResNet34_UNet_0.91.pth', help='ResNet34_UNet')
    parser.add_argument('--data_path', type=str, default='./dataset/oxford-iiit-pet', help='path to the input data')
    parser.add_argument('--batch_size', '-b', type=int, default=1, help='batch size')
    parser.add_argument('--model_path', type=str, default="./saved_models/model_UNet0.91pth", help='The model path ')
    parser.add_argument('--model', type=str, default="UNet", choices=['UNet', 'ResNet34_UNet'], help='UNet or ResNet34_UNet ')
    return parser.parse_args()
```



#其中 model_path 是填寫model的位置

```
python ./src/inference.py --model ResNet34_UNet --model_path "/saved_models/DL_Lab3_ ResNet34_UNet_313551110_陳憲驥.pth"

python ./src/inference.py --model UNet --model_path "/saved_models/DL_Lab3_ UNet_313551110_陳憲驥.pth"
```

6. Discussion

A. What architecture may bring better results?

- Deeper or Wider Networks**
 - Increase Network Depth:** Adding more layers to the network allows it to learn more high-level features. Deep networks like ResNet (Residual Networks) are common choices.
 - Increase Network Width:** Expanding the number of neurons in each layer can enhance the model's expressive power. Wide ResNet is an example of this approach.

- **Dense Connections:** As proposed in DenseNet, connect each layer to every other layer to improve feature reuse.
- **Custom Loss Functions**
 - Maybe we can try dice loss to improve model performance

B.What are the potential research topics in this task?

- Different data augmentation techniques: Using various data augmentation methods can lead to different results. We can further explore this by implementing more complex data augmentation.

ref : <https://github.com/GohVh/resnet34-unet>